# Leveraging Graph Neural Networks to Uncover Illicit Financial Networks

*A Comparative Study of GNN Architectures for Semi-Supervised Anomaly Detection on the Elliptic Bitcoin Dataset*

**Kuldip Manvar** & **Kartikey Tomar**

Indian Institute of Technology Kanpur

May 20, 2026

# Contents

# 1 Executive Summary

Financial crime detection has traditionally relied on rule-based systems and tabular machine learning models such as XGBoost, which treat each transaction independently. These approaches ignore the rich relational structure inherent in financial networks, where illicit actors exploit multi-hop transaction chains (e.g., smurfing, peeling chains) to obscure the provenance of funds.

In this project, we frame illicit transaction detection as a **node classification problem on a directed transaction graph** and benchmark several Graph Neural Network (GNN) architectures against an XGBoost baseline on the **Elliptic Bitcoin Dataset**. Our pipeline includes:

1. An **XGBoost baseline** (all 165 features) achieving PR-AUC of 0.708 and Illicit F1 of 0.758.
2. Three residual GNN architectures—**ResGCN**, **ResGAT**, and **ResGraphSAGE**—trained with focal loss, cosine-annealed learning rate, and early stopping.
3. A **GraphSAGE Ensemble** (3 models with different seeds) and a **94-feature ablation** (local features only) to test whether GNNs can replace hand-crafted structural features.
4. A **Rolling-Window + Replay Buffer** strategy to evaluate temporal robustness against concept drift.
5. **GNNExplainer**-based interpretability analysis on true-positive illicit nodes.

Our key finding is that while the XGBoost baseline dominates on aggregate metrics (PR-AUC 0.708, Macro F1 0.873), the rolling-window GraphSAGE variant achieves the highest illicit recall (0.788), making it more suitable for high-sensitivity regulatory settings. The SAGE 94-feature ablation (PR-AUC 0.550) demonstrates that GNNs can partially recover structural information from topology alone, though a gap remains.

# 2 Introduction and Motivation

## 2.1 Business Context

The modern financial ecosystem is increasingly digital, fast-paced, and borderless. Financial crimes—from credit card fraud and synthetic identity creation to large-scale money laundering—have evolved accordingly. Fraud is rarely perpetrated by isolated actors; it is driven by sophisticated, coordinated rings that utilize complex network topologies to obfuscate the source and destination of illicit funds.

Anti-money laundering (AML) compliance costs the global banking sector over $274 billion annually, yet traditional systems flag enormous volumes of false positives, exhausting investigator bandwidth while allowing genuinely illicit transactions to slip through.

## 2.2 Limitations of Traditional Approaches

Current AML and fraud detection systems rely predominantly on rule-based engines or tabular ML algorithms (e.g., Random Forests, XGBoost). These approaches suffer from several structural limitations:

- **The I.I.D. Assumption:** Traditional models assume data points are Independent and Identically Distributed. They evaluate a transaction based solely on its inherent features (amount, timestamp, fee) while completely ignoring network context.
- **Feature Engineering Bottlenecks:** To capture relational data in tabular models, data scientists must manually engineer features (e.g., "number of transactions with flagged accounts in the last 24 hours"). This is rigid and fails to capture deep, multi-hop structures.

- **High False Positive Rates:** Because tabular models lack contextual awareness, they frequently flag legitimate but unusual transactions, creating customer friction and operational waste.

### 2.3 Proposed Solution: Graph Neural Networks

We propose framing fraud detection as a **graph representation learning** problem. By defining the financial ecosystem as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of transaction nodes and $\mathcal{E}$ the set of payment flow edges, we can apply GNNs that operate via a **message-passing** framework: each node iteratively updates its representation by aggregating features from its neighbors. By stacking multiple layers, the model learns embeddings that encapsulate both local attributes and broader structural roles, effectively spotlighting sub-graphs typical of fraud rings.

## 3 Dataset: The Elliptic Bitcoin Dataset

### 3.1 Overview

The Elliptic Dataset is the benchmark for graph-based anomaly detection in cryptocurrency transaction networks. It was released by Elliptic, a blockchain analytics company, in collaboration with researchers from MIT and IBM.

Table 1: Elliptic Dataset Summary Statistics

| **Property** | **Value** |
|---|---|
| Domain | Bitcoin transaction flows |
| Total nodes (transactions) | 203,769 |
| Total edges (payment flows) | 234,355 |
| Graph type | Directed, homogeneous |
| Temporal time steps | 49 (≈2 weeks each) |
| Feature dimensions per node | 166 (1 timestep + 165 features) |
| – Local features | 94 (fee, volume, time, etc.) |
| – Aggregated structural features | 72 (hand-crafted neighborhood stats) |
| Labeled *licit* (class "2") | 42,019 (21%) |
| Labeled *illicit* (class "1") | 4,545 (2%) |
| Unknown / unlabeled | 157,205 (77%) |

### 3.2 Temporal Structure and Class Imbalance

The dataset spans 49 discrete time steps, each representing roughly two weeks of real-world Bitcoin transaction activity. The extreme class imbalance—only 2% of nodes are labeled illicit, 21% licit, and 77% unlabeled—makes this a challenging **semi-supervised** learning problem. Standard accuracy is an invalid metric in this setting; a model predicting all nodes as licit achieves 98%+ accuracy trivially.

### 3.3 Feature Space

Each node possesses 166 dimensions:

- **94 local features** (indices 1–94): directly derived from the transaction itself, including transaction fee, output volume, number of inputs/outputs, and temporal indicators.

- **72 aggregated structural features** (indices 95–165): hand-crafted neighborhood statistics that pre-encode relational information. A key experiment in our pipeline strips these 72 features to test whether GNNs can learn structural patterns purely from graph topology.

# 4 Data Preprocessing

## 4.1 Graph Construction

We construct a PyTorch Geometric `Data` object as follows:

1. **Node ID mapping:** Each unique `txId` is mapped to a contiguous integer index $\{0, 1, \ldots, N-1\}$ where $N = 203{,}769$.
2. **Label encoding:** Class "1" (illicit) $\rightarrow 1$, class "2" (licit) $\rightarrow 0$, "unknown" $\rightarrow -1$ (masked during loss computation).
3. **Edge index:** The edge list is filtered to retain only edges whose both endpoints exist in the node set. For GCN-based models, edges are converted to undirected via `to_undirected()`.

## 4.2 Temporal Train/Val/Test Split

To prevent temporal leakage and evaluate robustness to concept drift, we adopt a **strictly chronological** split:

Table 2: Temporal split of the Elliptic dataset

| Split | Time Steps | Labeled Licit | Labeled Illicit |
|---|---|---|---|
| Train | 1–34 | 29,044 | 3,569 |
| Validation | 35–38 | 1,325 | 259 |
| Test | 39–49 | 11,650 | 717 |

## 4.3 Leak-Free Feature Scaling

A `StandardScaler` is fitted *exclusively* on the training split and applied to all splits. This avoids information leakage from future time steps into the normalization statistics—a critical but frequently overlooked detail in graph ML pipelines.

## 4.4 Feature Ablation

We also construct a second data variant using only the 94 local features (stripping the 72 aggregated structural features). This ablation isolates the GNN's ability to learn relational patterns from graph topology alone, rather than relying on pre-computed neighborhood aggregates.

# 5 Methodology

## 5.1 Baseline: XGBoost

We train an XGBoost classifier on all 165 features as a strong tabular baseline. The model uses the features directly without graph structure. A threshold of 0.880 is selected via validation Macro F1 optimization.

## 5.2 GNN Architectures

All GNN models share a common design pattern—**residual connections**, **batch normalization**, and **dropout**—differing only in the message-passing operator. We describe each variant below.

### 5.2.1 ResGCN (Graph Convolutional Network)

GCN [1] aggregates messages via a symmetric normalized adjacency:

$$\mathbf{H}^{(\ell+1)} = \sigma\Big(\hat{\mathbf{D}}^{-1/2}\hat{\mathbf{A}}\hat{\mathbf{D}}^{-1/2}\mathbf{H}^{(\ell)}\mathbf{W}^{(\ell)}\Big) \tag{1}$$

where $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency with self-loops and $\hat{\mathbf{D}}$ is the corresponding degree matrix. Our ResGCN wraps each GCN layer with a residual skip connection:

$$\mathbf{h}_v^{(\ell+1)} = \mathrm{BN}\Big(\mathrm{GCNConv}(\mathbf{h}_v^{(\ell)}) + \mathbf{h}_v^{(\ell)}\Big) \tag{2}$$

### 5.2.2 ResGAT (Graph Attention Network)

GAT [2] replaces fixed aggregation weights with learned attention coefficients:

$$\alpha_{ij} = \frac{\exp\Big(\mathrm{LeakyReLU}\Big(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_j]\Big)\Big)}{\sum_{k\in\mathcal{N}(i)}\exp(\mathrm{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_k]))} \tag{3}$$

Our ResGAT uses multi-head attention (4 heads) with residual connections and batch normalization at each layer.

### 5.2.3 ResGraphSAGE

GraphSAGE [3] is designed for **inductive learning** via neighborhood sampling. Instead of requiring the full graph at inference time, it samples a fixed-size neighborhood and aggregates via a learned function:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\Big(\mathbf{W}^{(\ell)} \cdot \mathrm{CONCAT}\Big(\mathbf{h}_v^{(\ell)},\ \mathrm{AGG}\Big(\{\mathbf{h}_u^{(\ell)} : u \in \mathcal{N}(v)\}\Big)\Big)\Big) \tag{4}$$

This is particularly suitable for fraud detection in production, where new transactions arrive continuously and full-graph retraining is infeasible.

## 5.3 Common Training Configuration

Table 3: Hyperparameters shared across all GNN models

| **Hyperparameter** | **Value** |
|---|---|
| Hidden dimension | 128 |
| Number of GNN layers | 3 |
| Dropout rate | 0.3 |
| Learning rate (initial) | $5 \times 10^{-4}$ |
| Optimizer | AdamW (weight decay $= 10^{-4}$) |
| LR scheduler | Cosine annealing ($T_{\max} = 400$) |
| Loss function | Focal Loss ($\gamma = 2$, $\alpha = 0.75$) |
| Max epochs | 400 |
| Early stopping patience | 30 (on validation Macro F1) |

### 5.4 Focal Loss

Standard binary cross-entropy is ineffective under severe class imbalance. We employ **Focal Loss** [4]:

$$\mathcal{L}_{\text{focal}} = -\alpha_t(1 - p_t)^\gamma \log(p_t) \tag{5}$$

where $p_t$ is the predicted probability of the true class, $\alpha_t$ is a class-balancing weight, and $\gamma = 2$ is the focusing parameter that down-weights easy (well-classified) examples.

### 5.5 Threshold Calibration

Rather than using a fixed 0.5 decision threshold, we sweep thresholds on the validation set to maximize Macro F1, then apply the calibrated threshold to the test set. This accounts for the highly skewed class distribution and is critical for fair comparison.

### 5.6 Ensemble Strategy

We train a 3-member GraphSAGE ensemble with seeds $\{42, 59, 76\}$, averaging their predicted probabilities before threshold calibration. This stabilizes variance inherent in GNN training on imbalanced graphs.

### 5.7 Rolling Window with Replay Buffer

To simulate a realistic production deployment where models must adapt to evolving fraud patterns, we implement a **rolling-window** training strategy:

1. The model is initially trained on time steps 1–34.
2. For each subsequent test window, the model is fine-tuned on recent time steps while retaining a replay buffer of earlier labeled examples to prevent catastrophic forgetting.
3. Predictions are aggregated across all rolling test windows.

This strategy tests the model's robustness to **concept drift**—the phenomenon where fraud tactics evolve over time.

### 5.8 GNNExplainer for Interpretability

We apply **GNNExplainer** [5] to three correctly identified illicit nodes to extract:

- The **top-5 most influential features** driving the illicit prediction.
- The **important edges** (neighbor connections) that the model attends to.

This provides analysts with actionable, interpretable evidence—a regulatory requirement in the financial domain.

# 6 Libraries Used

Table 4: Key libraries and their roles

| Library | Version | Purpose |
|---|---|---|
| PyTorch | 2.10.0+cu128 | Deep learning framework |
| PyTorch Geometric | (latest) | GNN layers, data structures, NeighborLoader |
| XGBoost | (latest) | Tabular baseline classifier |
| scikit-learn | (latest) | Metrics, preprocessing, StandardScaler |
| NumPy | (latest) | Numerical computation |
| Pandas | (latest) | Data manipulation |
| Matplotlib / Seaborn | (latest) | Visualization |
| tqdm | (latest) | Training progress bars |

# 7 Model Architecture Details

All three GNN models follow a unified template:

$$\underbrace{\text{Input}(d_{\text{in}})}_{\text{165 or 94 features}} \xrightarrow{\text{Linear}} \underbrace{\text{ResBlock}_1 \to \text{ResBlock}_2 \to \text{ResBlock}_3}_{\text{3 residual GNN layers, dim=128}} \xrightarrow{\text{Linear}} \underbrace{\text{Output}(1)}_{\text{logit}}$$

Each **ResBlock** consists of:

$$\mathbf{h}' = \text{Dropout}(\text{ELU}(\text{BatchNorm}(\text{GNNConv}(\mathbf{h}) + \mathbf{h})))$$

The final layer projects the 128-dimensional embedding to a single logit, passed through a sigmoid for binary classification. The residual skip connections are essential to prevent over-smoothing in deeper GNN architectures, where repeated message passing causes node embeddings to converge.

# 8 Experimental Results

## 8.1 Final Comparison Table

Table 5: Test set performance across all models. Best per column in **bold**.

| Model | PR-AUC | Macro F1 | Illicit F1 | Illicit Recall |
|---|---|---|---|---|
| XGBoost (baseline) | **0.708** | **0.873** | **0.758** | 0.622 |
| ResGCN | 0.548 | 0.769 | 0.562 | 0.519 |
| ResGAT | 0.406 | 0.696 | 0.418 | 0.317 |
| ResGraphSAGE | 0.392 | 0.725 | 0.479 | 0.420 |
| SAGE Ensemble (3×) | 0.526 | 0.742 | 0.505 | 0.360 |
| SAGE (94 local features) | 0.550 | 0.770 | 0.560 | 0.434 |
| ResGraphSAGE (Rolling) | 0.627 | 0.653 | 0.387 | **0.788** |

## 8.2 Analysis of Results

### 8.2.1 XGBoost Dominance on Aggregate Metrics

The XGBoost baseline achieves the highest PR-AUC (0.708) and Macro F1 (0.873) by a significant margin. This is attributable to two factors: (1) the 72 aggregated structural features in the Elliptic dataset were specifically engineered to encode neighborhood information, partially preempting the GNN's advantage; and (2) XGBoost is exceptionally well-suited for tabular data with hand-crafted features, operating without the over-smoothing and training instability issues that plague GNNs.

### 8.2.2 GNN Performance Hierarchy

Among GNN variants, **ResGCN** performs best overall (PR-AUC 0.548, Illicit F1 0.562), followed by **ResGraphSAGE** and then **ResGAT**. The underperformance of GAT is notable—its attention mechanism may be over-parameterized for this graph structure, leading to overfitting on the small illicit class.

### 8.2.3 Feature Ablation: GNNs Learn Structure

The SAGE model trained on only 94 local features achieves PR-AUC 0.550 and Illicit F1 0.560—*comparable to or slightly better than* the full-feature ResGCN (0.548 / 0.562). This demonstrates that GNNs can partially recover the information encoded in the hand-crafted structural features by learning directly from graph topology. However, a gap remains relative to XGBoost with all 165 features.

### 8.2.4 Rolling Window: Trading Precision for Recall

The rolling-window ResGraphSAGE achieves by far the highest illicit recall at **0.788**—catching nearly 79% of all illicit transactions—but at the cost of a lower Macro F1 (0.653) and higher false positive rate. This precision-recall trade-off is highly relevant in regulatory contexts where *missing* an illicit transaction (false negative) carries far greater cost than flagging a benign one (false positive).

## 8.3 Training Dynamics

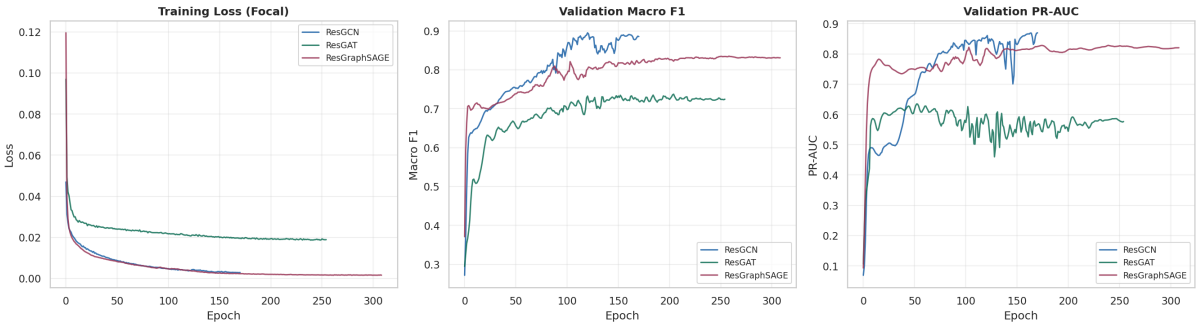


Figure 1: Training loss (focal), validation Macro F1, and validation PR-AUC across epochs for ResGCN, ResGAT, and ResGraphSAGE. ResGCN converges fastest and achieves the lowest training loss; ResGAT plateaus at a lower validation F1.

Key observations from the training curves:

- ResGCN achieves the lowest training loss and highest validation metrics, consistent with its test performance.

- ResGraphSAGE shows competitive validation PR-AUC (∼0.82) but a gap to test PR-AUC (0.39), suggesting overfitting to the validation time window.
- ResGAT converges to a higher loss plateau, with validation Macro F1 stabilizing around 0.65—well below the other two architectures.

## 8.4 Precision-Recall Curves

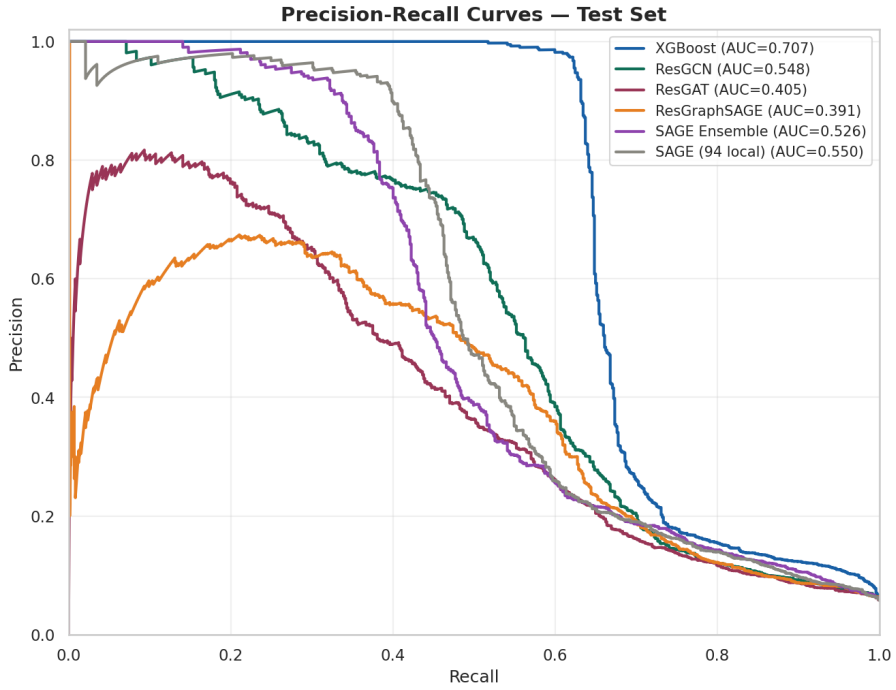


Figure 2: Precision-Recall curves on the test set. XGBoost dominates across nearly all recall levels. The SAGE (94 local) variant shows competitive precision at low recall, suggesting strong confidence on its top-ranked predictions.

## 8.5 Confusion Matrices

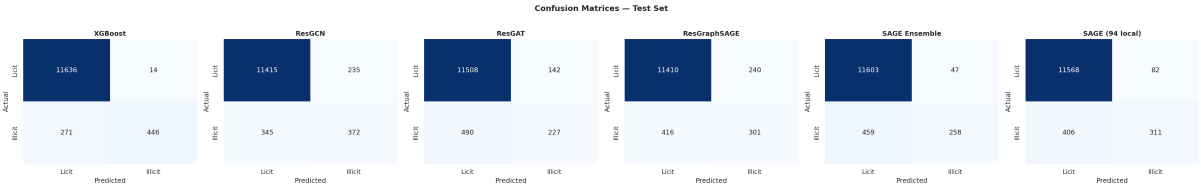


Figure 3: Confusion matrices for all models on the test set (11,650 licit + 717 illicit). XGBoost misclassifies only 271 illicit as licit (best), while ResGAT misses 490 (worst among static models).

## 8.6 GNNExplainer Results

GNNExplainer was applied to three true-positive illicit nodes from the ResGCN model:

Table 6: GNNExplainer output: top features and important edges for three illicit nodes

| Node ID | Top-5 Features | Important Edges |
|---|---|---|
| 154383 | f145, f3, f138, f6, f95 | 2 |
| 154384 | f154, f144, f65, f112, f164 | 2 |
| 154469 | f78, f61, f125, f145, f55 | 1 |

Feature `f145` appears in two of the three explained nodes, suggesting it is a strong discriminative signal for illicit activity. Notably, features from both the local range (f3, f6, f55, f61, f65, f78) and the aggregated structural range (f95, f112, f125, f138, f144, f145, f154, f164) appear, confirming that both feature groups contribute to the model's predictions.

# 9 Conclusions, Improvements, and Limitations

## 9.1 Key Conclusions

1. **XGBoost remains a formidable baseline** when the dataset includes hand-crafted structural features. The 72 aggregated features largely neutralize the GNN's topological advantage.
2. **GNNs can learn structure from topology**, as evidenced by the 94-feature SAGE ablation achieving competitive performance without any aggregated features.
3. **Rolling-window training maximizes illicit recall** (0.788), making it the preferred strategy for regulatory use cases where false negatives are costlier than false positives.
4. **ResGCN outperforms ResGAT and ResGraphSAGE** in the static transductive setting, likely because GCN's fixed aggregation is more stable on this graph structure.
5. **GNNExplainer provides actionable interpretability**, identifying specific features and edges driving predictions—a compliance requirement.

## 9.2 Improvements over Legacy Approaches

- Unlike traditional tabular models, GNNs encode multi-hop neighborhood context automatically, eliminating manual feature engineering.
- Focal loss and threshold calibration substantially improve minority-class detection compared to standard cross-entropy with default thresholds.
- The rolling-window approach addresses concept drift, a critical real-world challenge that static train/test splits ignore.
- GNNExplainer bridges the gap between black-box neural networks and the interpretability demanded by financial regulators.

## 9.3 Limitations

- **Generalization gap:** All GNN models show a significant validation-to-test performance drop, indicating sensitivity to temporal distribution shift beyond the training horizon.
- **Feature anonymization:** The Elliptic dataset features are anonymized, preventing domain-specific feature engineering and limiting interpretability of GNNExplainer outputs.
- **Scale:** With ∼200K nodes, the Elliptic dataset is relatively small. Production AML systems process billions of transactions; scalability via mini-batch training (GraphSAGE + NeighborLoader) would need further investigation.
- **Semi-supervised setting:** 77% of nodes are unlabeled. Incorporating these via self-training, label propagation, or contrastive pre-training could improve performance.

- **Directed graph underutilization:** Converting edges to undirected (for GCN) discards transaction flow directionality. Directed GNN variants (e.g., MagNet, Dir-GNN) may preserve this signal.

### 9.4 Future Work

- Temporal GNNs (TGN, DyRep) that natively model evolving graph snapshots.
- Hybrid GNN + XGBoost ensembles that combine topological embeddings with tabular features.
- Contrastive or self-supervised pre-training on the unlabeled 77% of nodes.
- Deployment of the rolling-window GraphSAGE model in a streaming inference pipeline with periodic model updates.

## 10 References

## References

[1] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *Proc. ICLR*, 2017.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *Proc. ICLR*, 2018.

[3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Proc. NeurIPS*, 2017.

[4] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. ICCV*, 2017.

[5] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating Explanations for Graph Neural Networks," in *Proc. NeurIPS*, 2019.

[6] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," in *KDD Workshop on Anomaly Detection in Finance*, 2019.

[7] A. Pareja et al., "EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs," in *Proc. AAAI*, 2020.

[8] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" in *Proc. ICLR*, 2019.